Proyecto Sistema de Gestión de Ventas

Evaluación módulo 7
Creación y Manipulación de Tablas

Nombre: Ruth Richards G

Tabla de contenido

I. INTRODUCCION 3

II. EXPLICACION DE LA ESTRUCTURA DEL PROYECTO 4

A. Lección 1: Las bases de datos relacionales. 4

Ventajas frente al uso de planillas: 4
Estructura mínima en SQLite con nombre y propósito del sistema. 5

B. Lección 2: Consultas a una sola tabla. 6

Qué hace esta lección: 6
Resultados obtenidos en esta lección: 7

C. Lección 3: Consultas a tablas relacionadas. 7

Qué hace esta lección: 7
Resultados obtenidos en esta lección: 7

D. Lección 4: Consultas agrupadas. 7

Qué hace esta lección: 7
Resultados obtenidos en esta lección: 8

E. Lección 5: Consultas anidadas. 8

Qué hace esta lección: 8
Resultados obtenidos en esta lección: 9

F. Lección 6: Creación y manipulación de tablas. 9

Qué hace esta lección: 9
Resultados obtenidos en esta lección: 10

III. CONCLUSION 11

IV. ANEXOS 12

A. Captura de pantalla de la ejecución en SQLiteOnline 12

Lección 2 12
Lección 3 13
Lección 4 14
Lección 5 15
Lección 6 17

B. Link del código subido a GitHub 18

I. INTRODUCCION

El proyecto final del módulo 7 solicita desarrollar un sistema básico de gestión de ventas para registrar, clientes, productos y transacciones en una base de datos relacional. Actualmente, la empresa lleva su información en planillas que generan errores frecuentes y dificultan el análisis posterior. El equipo de tecnología solicita el desarrollo de una solución simple en SQL que permita crear, poblar y consultar datos en un entorno confiable para la empresa comercial “Comercio Ya”.

El objetivo del proyecto es desarrollar un sistema de gestión de ventas utilizando una base de datos relacional, permitiendo almacenar, consultar y manipular información de clientes, productos y ventas. El sistema debe estar completamente funcional, con su estructura deseñado y sus consultas básicas operativas, utilizando únicamente los contenidos de los manuales estudiados en el módulo.

II. EXPLICACION DE LA ESTRUCTURA DEL PROYECTO

A. Lección 1: Las bases de datos relacionales.

Una base de datos relacional es un sistema que organiza datos en tablas relacionadas entre sí mediante claves.

En este modelo, cada tabla representa una entidad del negocio: Clientes, Productos y Ventas.

Cada tabla contiene filas que representan cada elemento y columnas que representan sus atributos. Las relaciones se establecen mediante clave primaria que identifica de forma única cada registro y foránea que conecta una tabla con otra. En este proyecto la tabla ventas conecta a client4es con productos, permitiendo registrar las transacciones.

En la "**Introducción**" se señaló que la empresa **"Comercio Ya"** necesitaba de dejar de usar plantillas y pasar a un sistema estructurado. El el diagrama definido el sistema permitirá registrar clientes con sus datos, administrar productos, registrar ventas con información detallada, analizar el comportamiento de compra, entre otros.

Ventajas frente al uso de planillas:

Eliminación de duplicidad, por ejemplo el cliente se almacena una sola vez en la tabla de clientes.

Integridad de datos, gracias a las relaciones se reducen errores humanos, no se puede registrar una venta sin cliente o producto válido.

Mejor organización, lo que permite mantener la información ordenada y escalable. Separa claramente Clientes, Productos, Ventas.

Escalabilidad, permite agregar más clientes sin afectar las ventas, se puede incorporar nuevos productos fácilmente y expandir el sistema a futuro.

Estructura mínima en SQLite con nombre y propósito del sistema.

El sistema se denomina: Sistema de Gestión de Ventas, Comercio Ya

El propósito es gestionar la información de clientes, productos, ventas mediante una base de datos relacional que permita almacenar, consultas y analizar datos de manera eficiente.

TABLAS DEL SISTEMA		
CLIENTES	**PRODUCTOS**	**VENTAS**
Cliente_id	Producto_id	Ventas_id
Nombres	Nombre_producto	Cliente_id
Ciudad	Categoria	Producto_id
Region	Precio_unitario	Fecha
Genero	Fecha_ingreso	Cantidad
Telefono	Stock	Impuesto
Email		Descuento
Fecha_registro		Total_venta

La Tabla ventas se relaciona con clientes y productos mediante claves foráneas, estableciendo una relación de uno a muchos, donde un cliente puede realizar múltiples compras y un producto puede estar presente en múltiples ventas.

Query Historial

```sql
-- 1. ELIMINAR TABLA SI YA EXISTE (Para evitar errores de "Table already exists")
DROP TABLE IF EXISTS Clientes;

-- 2. CREAR LA TABLA CLIENTES
CREATE TABLE Clientes (
    cliente_id INTEGER PRIMARY KEY,
    nombres TEXT NOT NULL,
    ciudad TEXT NOT NULL,
    region TEXT NOT NULL,
    genero TEXT,
    telefono TEXT,
    email TEXT,
    fecha_registro DATE
);
```

Query Historial

```sql


CREATE TABLE IF NOT EXISTS Productos (
    producto_id INTEGER PRIMARY KEY,
    nombre_producto TEXT NOT NULL,
    categoria TEXT,
    precio_unitario REAL,
    fecha_ingreso DATE
);


CREATE TABLE IF NOT EXISTS Ventas (
    venta_id INTEGER PRIMARY KEY,
    cliente_id INTEGER,
    producto_id INTEGER,
    fecha_venta DATE,
    cantidad INTEGER,
    impuesto REAL,
    descuento REAL,
    total_venta REAL,
```

Se crea la tabla “Clientes” la que almacenará la información de los clientes del sist4ema. La clave primaria cliente_id permite identificar de forma única cada registro, asegurando integridad en los datos.

B. Lección 2: Consultas a una sola tabla.

Qué hace esta lección:

Permitió comprender cómo recuperar información desde una tabla, utilizando filtros básicos. Esto es fundamental para cualquier sistema, ya que permite acceder rápidamente a los datos almacenados.

```sql
-- Realizar consultas simples para seleccionar:
--  todos los clientes
--SELECT * FROM Clientes

--  filtrar por ciudad
--SELECT * FROM Clientes WHERE ciudad = "La Serena"

--  filtrar por nombre
--SELECT * FROM Clientes WHERE nombres like "%Mariana%"
```

Resultados obtenidos en esta lección:

Creación de la tabla clientes con sus respectivos campos, inserción de registros de prueba, ejecución de consultas simples utilizando SELECT. Esto permitió visualizar todos los registros, filtrar clientes por ciudad, buscar clientes por nombre.

C. Lección 3: Consultas a tablas relacionadas.

Qué hace esta lección:

En esta lección se crearon las tablas productos y ventas, y se establecieron relaciones mediante claves foráneas. La tabla ventas actúa como una tabla que se conecta con las tablas clientes y productos.

El uso de JOIN permite combina datos de múltiples tablas, lo que es esencial en bases de datos relacionales. Gracias a esto, se puede obtener información completa sin duplicar datos.

```
-- Ejecutar consultas con JOIN para obtener: qué cliente compró qué producto y cuándo.
/*
SELECT v.venta_id, v.fecha, c.nombres, p.nombre_producto, v.cantidad, v.total_venta
FROM Ventas As v
    LEFT JOIN Clientes As c ON c.cliente_id = v.cliente_id
    LEFT JOIN Productos As p ON p.producto_id = v.producto_id
*/
```

Resultados obtenidos en esta lección:

Dentro de los resultados obtenidos está, el identificar qué cliente compró un producto, determinar la fecha de compra del producto.

D. Lección 4: Consultas agrupadas.

Qué hace esta lección:

Las consultas agrupadas permiten transformar datos individuales en información útil para la toma de decisiones, facilitando el análisis del comportamiento de clientes y productos.

En esta lección se utilizaron funciones como: SUM, COUNT y AVG, se realizaron agrupaciones con GROUP BY, que permitió analizar la información de forma más resumida.

```
-- Usar funciones como SUM(), AVG(), COUNT() sobre la tabla de ventas.
SELECT COUNT(*) AS numero_facturas,
    SUM(total_venta) AS total_vendido,
    SUM(total_venta) / COUNT(*) AS ticket_promedio
FROM Ventas;


-- Agrupar ventas por cliente y por producto.
SELECT c.nombres, p.nombre_producto,
    COUNT(*) As numero_compras, SUM(v.total_venta) As total_ventas
FROM Ventas As v
    LEFT JOIN Clientes As c ON c.cliente_id = v.cliente_id
    LEFT JOIN Productos As p ON p.producto_id = v.producto_id
GROUP BY c.nombres, p.nombre_producto;
```

Resultados obtenidos en esta lección:

Entre los resultados obtenidos tenemos el total de compras por cliente, cantidad de ventas por producto, promedio de ventas en datos individuales.

E. Lección 5: Consultas anidadas.

Qué hace esta lección:

En esta lección se utilizaron subconsultas para responder preguntas más avanzadas del negocio como: Qué Clientes han realizado más de una compra, Cuál ha sido el producto más vendido, Qué Cliente ha gastado más dinero. Por ello, las consultas anidadas permite resolver consultas más complejas mediante subconsultas.

```sql
Lección 5: Consultas anidadas
*/

-- Listar los clientes que hayan hecho más de una compra.
-- Que tenga al menos 3 compras

/*
SELECT nombres
FROM Clientes
WHERE cliente_id IN (SELECT cliente_id
                        FROM Ventas
                        GROUP BY cliente_id
                            HAVING COUNT(*) >= 3
                    )
*/


-- Obtener el producto más vendido utilizando una subconsulta.
/*
SELECT nombre_producto
FROM Productos
WHERE producto_id IN (
                    SELECT v.producto_id
                        FROM Ventas AS v
                            LEFT JOIN Productos AS p ON p.producto_id = v.producto_id
                        GROUP BY v.producto_id
                        ORDER BY SUM(cantidad) DESC
                        LIMIT 1
                )
*/

-- Versión 2 - Multi modal
/*
SELECT v.producto_id, p.nombre_producto, SUM(v.cantidad) AS cant
    FROM Ventas AS v
        LEFT JOIN Productos AS p ON p.producto_id = v.producto_id
    GROUP BY v.producto_id, p.nombre_producto
        HAVING SUM(v.cantidad) = (SELECT MAX(cant) AS cant
                                    FROM (
                                            SELECT producto_id, COUNT(*) As cant
                                            FROM Ventas
                                            GROUP BY producto_id
                                            )
                                    )
*/

-- Consultar el cliente que más gastó en total.
SELECT v.cliente_id, c.nombres, SUM(v.total_venta) AS total
    FROM Ventas AS v
        LEFT JOIN Clientes AS c ON c.cliente_id = v.cliente_id
    GROUP BY v.cliente_id, c.nombres
        HAVING SUM(v.total_venta) = (SELECT MAX(total) AS total
                                    FROM (
                                            SELECT cliente_id, SUM(total_venta) As total
                                            FROM Ventas
                                            GROUP BY cliente_id
                                            )
                                    )
```

Resultados obtenidos en esta lección:

Los resultados obtenidos en las subconsultas, permitieron utilizar el resultado de una consulta dentro de otra, lo que facilita la obtención de información más específica y compleja. Esto fue clave para el análisis más profundo en la base de datos.

F. Lección 6: Creación y manipulación de tablas.

Qué hace esta lección:

Esta lección modifica y administra tablas mediante comandos SQL, como comandos de manipulación de datos y definición de datos: ALTER TABLE; UPDATE; DELETE.

Con ello, se pudo comprender cómo mantener actualizada la base de datos, adaptándola a cambios del negocio. La manipulación de datos es fundamental para asegurar la vigencia y la calidad de la información.

```
/*
Lección 6: Creación y manipulación de tablas
*/

-- Modificar la tabla de productos para agregar una columna stock.
--ALTER TABLE Productos ADD COLUMN stock INTEGER
--SELECT * FROM Productos
-- Reemplazar valores null
/*
SELECT producto_id, nombre_producto, categoria, precio_unitario, fecha_ingreso,
    IFNULL(stock, 10) AS stock
FROM Productos
*/

-- Grabar en la tabla el valor inicial del stock
/*
UPDATE Productos
SET stock = 0
WHERE stock IS NULL
*/

-- Actualizar el stock luego de una venta.

-- 1. Necesitamos stock inicial
--UPDATE Productos SET stock = 10 WHERE producto_id = 1
--UPDATE Productos SET stock = 20 WHERE producto_id = 2
--UPDATE Productos SET stock = 30 WHERE producto_id = 3

-- 2. Registrar venta
/*
INSERT INTO Ventas
(venta_id, cliente_id, producto_id, fecha, cantidad, total_venta)
VALUES
(8, 3, 3, '2026-03-15', 1, 25000)
*/

-- 3. Actualizar stock
/*
UPDATE Productos
SET stock = stock - 1
WHERE producto_id = 3
*/

-- Eliminar un producto obsoleto y documentar el impacto.
-- NO VA
```

Resultados obtenidos en esta lección:

Dentro de los resultados obtenidos están: Campo Stock agregado a la tabla de productos, actualización del stock después de que se genere una venta, eliminación de producto obsoleto.

III. CONCLUSION

El proyecto final del módulo 7 permitió diseñar una base de datos relacional funcional para la gestión de ventas, integrando clientes, productos y transacciones para la empresa “Comercio Ya”. A partir del uso de SQL, se logró almacenar, consultar y analizar información de manera oportuna y eficiente, demostrando la importancia de las bases de datos en la toma de decisiones y en la mejora de los procesos en la empresa.

A modo personal, el desarrollo de este proyecto me ayudó a comprender la importancia de estructurar correctamente la información desde el primer momento. Al trabajar con base de datos relacionales, logré entender cómo el diseño impacta en el análisis y la toma de decisiones.

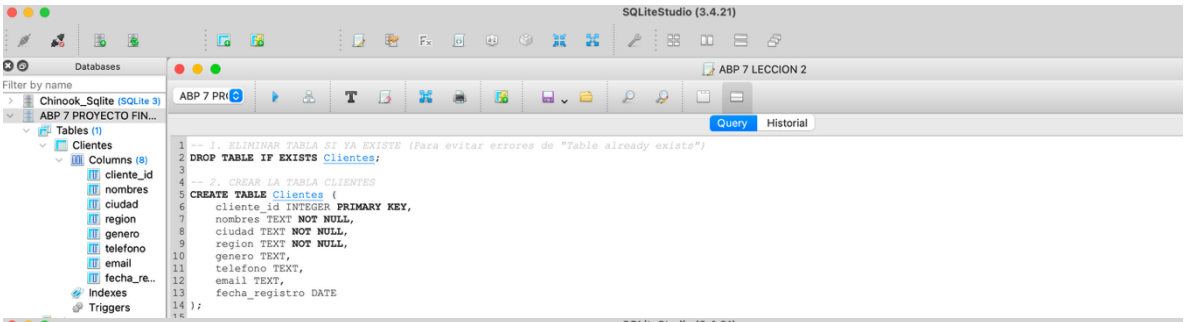
Visualizar esta herramienta y como puede aplicarse en contextos reales, en este caso en áreas de gestión comercial y financiera, donde la información es clave para la toma oportuna de decisiones.

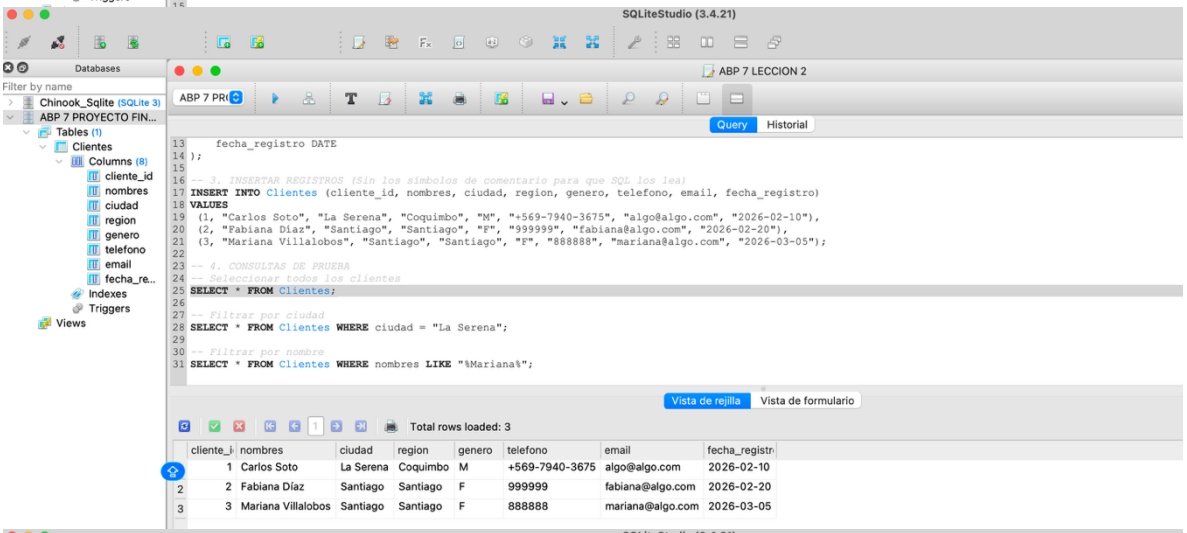
Este aprendizaje, sin duda, representa una base para seguir profundizando en el análisis de datos.

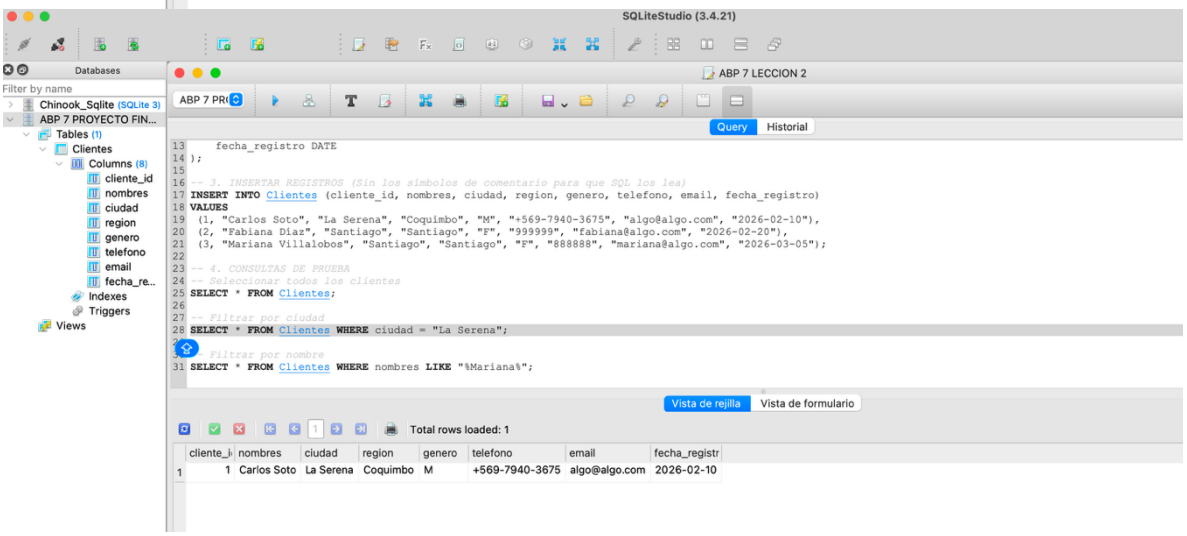
IV. ANEXOS

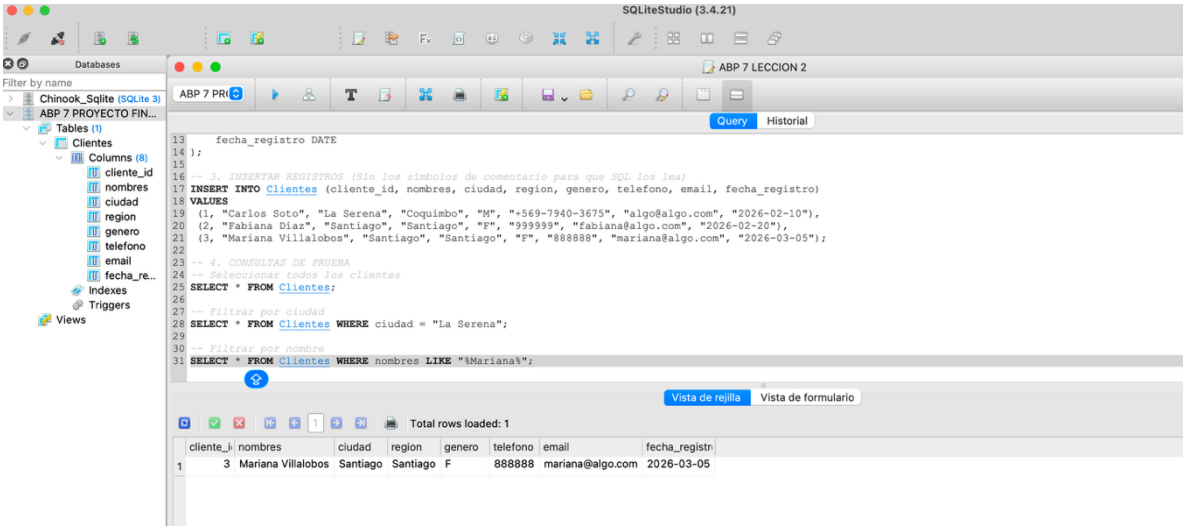
A. Captura de pantalla de la ejecución en SQLiteOnline

Lección 2

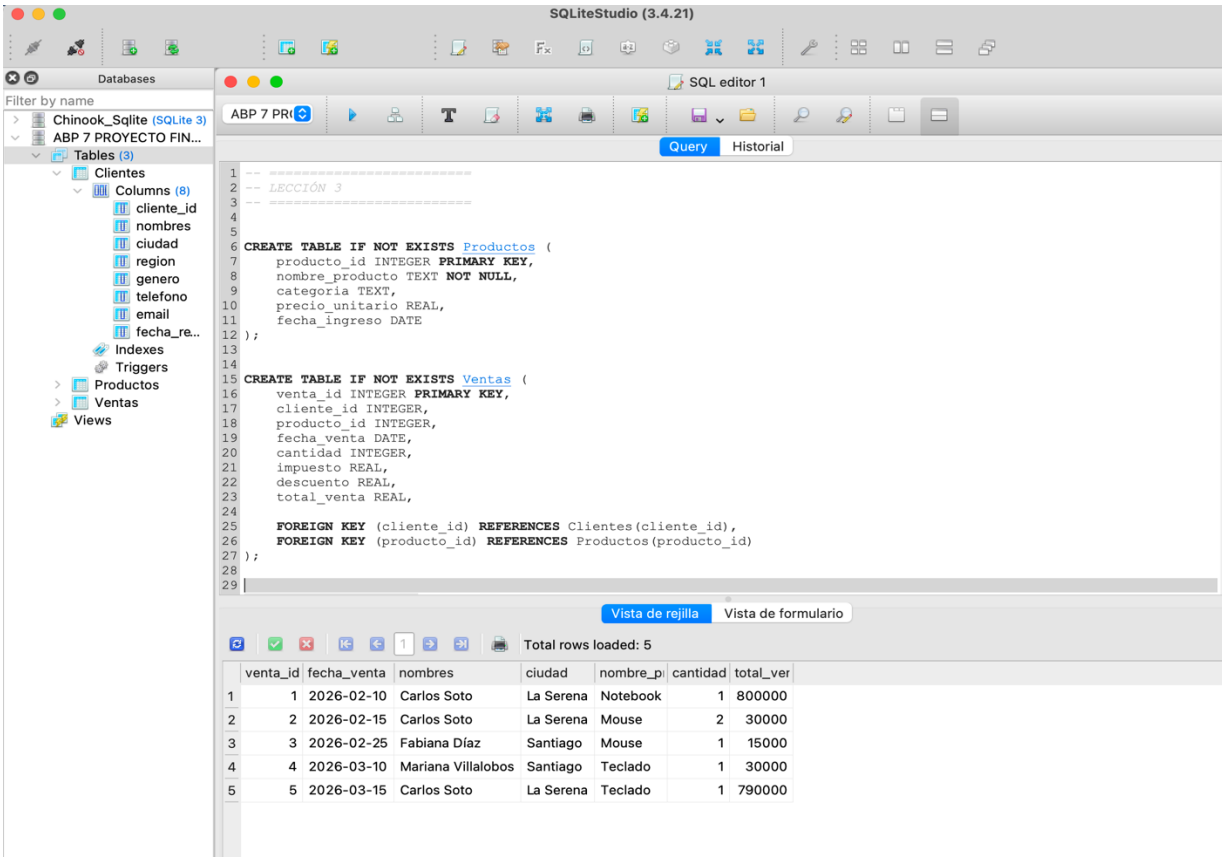




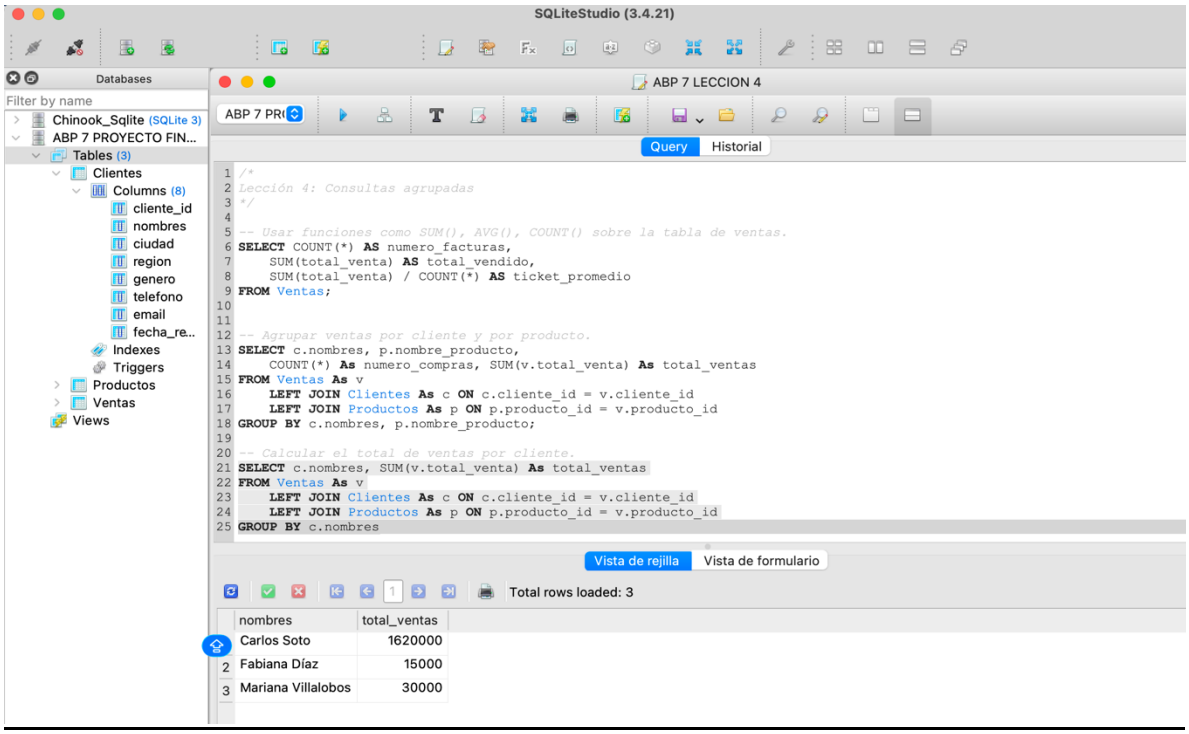



SQLiteStudio (3.4.21)
Databases
Filter by name
Chinook_Sqlite (SQLite 3)
ABP 7 PROYECTO FIN...
Tables (1)
Clientes
Columns (8)
cliente_id
nombres
ciudad
region
genero
telefono
email
fecha_re...
Indexes
Triggers
Views
ABP 7 LECCION 2
Query
Historial
fecha_registro DATE
);
-- 3. INSERTAR REGISTROS (Sin los simbolos de comentario para que SQL los lea)
INSERT INTO Clientes (cliente_id, nombres, ciudad, region, genero, telefono, email, fecha_registro)
VALUES
(1, "Carlos Soto", "La Serena", "Coquimbo", "M", "+569-7940-3675", "algo@algo.com", "2026-02-10"),
(2, "Fabiana Díaz", "Santiago", "Santiago", "F", "999999", "fabiana@algo.com", "2026-02-20"),
(3, "Mariana Villalobos", "Santiago", "Santiago", "F", "888888", "mariana@algo.com", "2026-03-05");
-- 4. CONSULTAS DE PRUEBA
-- Seleccionar todos los clientes
SELECT * FROM Clientes;
-- Filtrar por ciudad
SELECT * FROM Clientes WHERE ciudad = "La Serena";
-- Filtrar por nombre
SELECT * FROM Clientes WHERE nombres LIKE "%Mariana%";
Vista de rejilla
Vista de formulario
Total rows loaded: 1
cliente_i nombres ciudad region genero telefono email fecha_registr
1 3 Mariana Villalobos Santiago Santiago F 888888 mariana@algo.com 2026-03-05

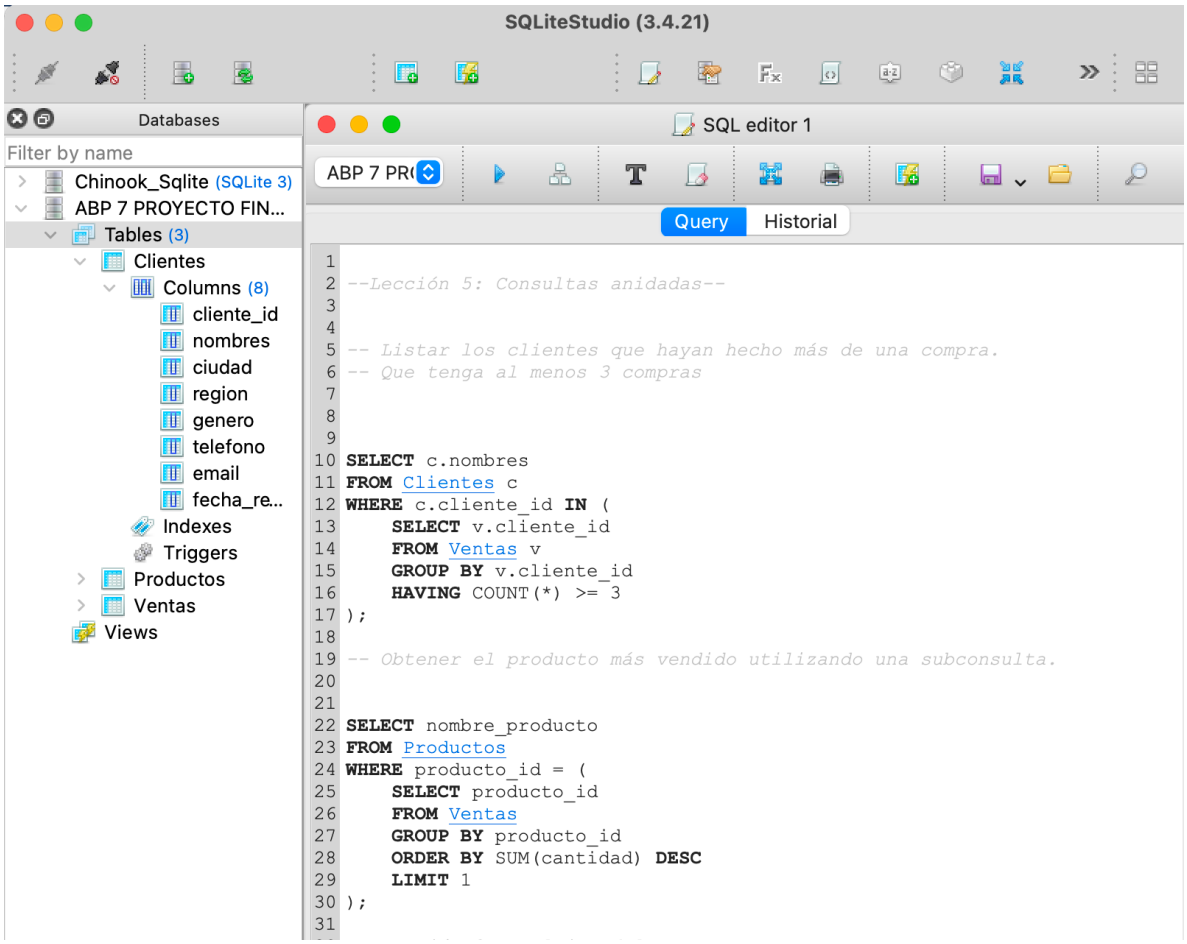
Lección 3


SQLiteStudio (3.4.21)
Databases
Filter by name
Chinook_Sqlite (SQLite 3)
ABP 7 PROYECTO FIN...
Tables (3)
Clientes
Columns (8)
cliente_id
nombres
ciudad
region
genero
telefono
email
fecha_re...
Indexes
Triggers
Productos
Ventas
Views
SQL editor 1
Query
Historial
-- ==========
-- LECCIÓN 3
-- ==========
CREATE TABLE IF NOT EXISTS Productos (
producto_id INTEGER PRIMARY KEY,
nombre_producto TEXT NOT NULL,
categoria TEXT,
precio_unitario REAL,
fecha_ingreso DATE
);
CREATE TABLE IF NOT EXISTS Ventas (
venta_id INTEGER PRIMARY KEY,
cliente_id INTEGER,
producto_id INTEGER,
fecha_venta DATE,
cantidad INTEGER,
impuesto REAL,
descuento REAL,
total_venta REAL,
FOREIGN KEY (cliente_id) REFERENCES Clientes(cliente_id),
FOREIGN KEY (producto_id) REFERENCES Productos(producto_id)
);
Vista de rejilla
Vista de formulario
Total rows loaded: 5
venta_id fecha_venta nombres ciudad nombre_p cantidad total_ver
1 1 2026-02-10 Carlos Soto La Serena Notebook 1 800000
2 2 2026-02-15 Carlos Soto La Serena Mouse 2 30000
3 3 2026-02-25 Fabiana Díaz Santiago Mouse 1 15000
4 4 2026-03-10 Mariana Villalobos Santiago Teclado 1 30000
5 5 2026-03-15 Carlos Soto La Serena Teclado 1 790000

Lección 4


SQLiteStudio (3.4.21)
Databases
Filter by name
Chinook_Sqlite (SQLite 3)
ABP 7 PROYECTO FIN...
Tables (3)
Clientes
Columns (8)
cliente_id
nombres
ciudad
region
genero
telefono
email
fecha_re...
Indexes
Triggers
Productos
Ventas
Views
ABP 7 LECCION 4
ABP 7 PR
Query
Historial
/*
Lección 4: Consultas agrupadas
*/
-- Usar funciones como SUM(), AVG(), COUNT() sobre la tabla de ventas.
SELECT COUNT(*) AS numero_facturas,
SUM(total_venta) AS total_vendido,
SUM(total_venta) / COUNT(*) AS ticket_promedio
FROM Ventas;
-- Agrupar ventas por cliente y por producto.
SELECT c.nombres, p.nombre_producto,
COUNT(*) As numero_compras, SUM(v.total_venta) As total_ventas
FROM Ventas As v
LEFT JOIN Clientes As c ON c.cliente_id = v.cliente_id
LEFT JOIN Productos As p ON p.producto_id = v.producto_id
GROUP BY c.nombres, p.nombre_producto;
-- Calcular el total de ventas por cliente.
SELECT c.nombres, SUM(v.total_venta) As total_ventas
FROM Ventas As v
LEFT JOIN Clientes As c ON c.cliente_id = v.cliente_id
LEFT JOIN Productos As p ON p.producto_id = v.producto_id
GROUP BY c.nombres
Vista de rejilla
Vista de formulario
Total rows loaded: 3
nombres
total_ventas
Carlos Soto
1620000
2
Fabiana Díaz
15000
3
Mariana Villalobos
30000

Lección 5


SQLiteStudio (3.4.21)
Databases
Filter by name
Chinook_Sqlite (SQLite 3)
ABP 7 PROYECTO FIN...
Tables (3)
Clientes
Columns (8)
cliente_id
nombres
ciudad
region
genero
telefono
email
fecha_re...
Indexes
Triggers
Productos
Ventas
Views
SQL editor 1
ABP 7 PR(
Query
Historial
--Lección 5: Consultas anidadas--
-- Listar los clientes que hayan hecho más de una compra.
-- Que tenga al menos 3 compras
SELECT c.nombres
FROM Clientes c
WHERE c.cliente_id IN (
SELECT v.cliente_id
FROM Ventas v
GROUP BY v.cliente_id
HAVING COUNT(*) >= 3
);
-- Obtener el producto más vendido utilizando una subconsulta.
SELECT nombre_producto
FROM Productos
WHERE producto_id = (
SELECT producto_id
FROM Ventas
GROUP BY producto_id
ORDER BY SUM(cantidad) DESC
LIMIT 1
);

SQLiteStudio (3.4.21)

Databases

Filter by name

- Chinook_Sqlite (SQLite 3)
- ABP 7 PROYECTO FIN...
  - Tables (3)
    - Clientes
      - Columns (8)
        - cliente_id
        - nombres
        - ciudad
        - region
        - genero
        - telefono
        - email
        - fecha_re...
      - Indexes
      - Triggers
    - Productos
    - Ventas
  - Views

SQL editor 1

ABP 7 PR(

Query | Historial

```
28     ORDER BY SUM(cantidad) DESC
29     LIMIT 1
30 );
31
32 -- Versión 2 - Multi modal
33
34
35 SELECT v.producto_id, p.nombre_producto, SUM(v.cantidad) AS cant
36 FROM Ventas v
37 JOIN Productos p ON p.producto_id = v.producto_id
38 GROUP BY v.producto_id, p.nombre_producto
39 HAVING SUM(v.cantidad) = (
40     SELECT MAX(total_cantidad)
41     FROM (
42         SELECT producto_id, SUM(cantidad) AS total_cantidad
43         FROM Ventas
44         GROUP BY producto_id
45     )
46 );
47
```

Vista de rejilla | Vista de formulario

Total rows loaded: 1

	producto_id	nombre_producto	cant
1	2	Mouse	3

SQLiteStudio (3.4.21)

Databases

Filter by name

- Chinook_Sqlite (SQLite 3)
- ABP 7 PROYECTO FIN...
  - Tables (3)
    - Clientes
      - Columns (8)
        - cliente_id
        - nombres
        - ciudad
        - region
        - genero
        - telefono
        - email
        - fecha_re...
      - Indexes
      - Triggers
    - Productos
    - Ventas
  - Views

SQL editor 1

ABP 7 PR(

Query | Historial

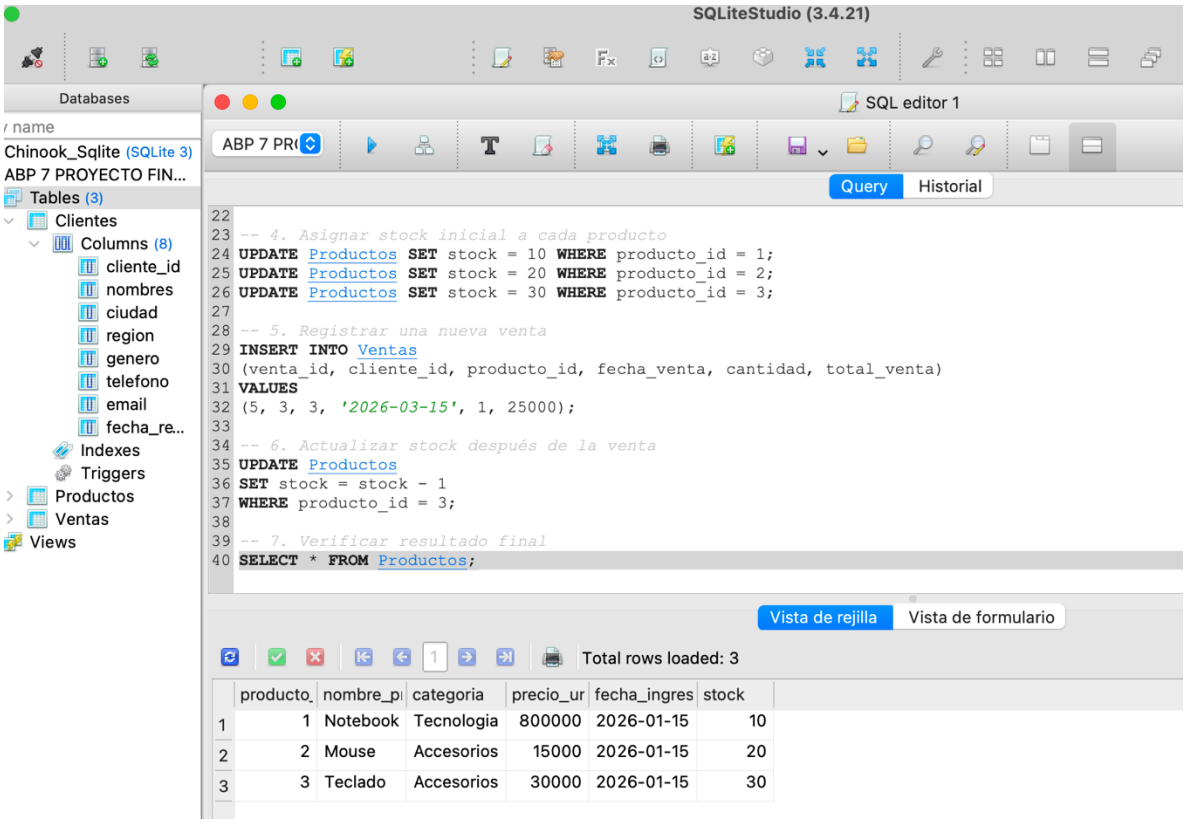
```
46 );
47
48 -- Consultar el cliente que más gastó en total.
49
50 SELECT v.cliente_id, c.nombres, SUM(v.total_venta) AS total
51 FROM Ventas AS v
52 LEFT JOIN Clientes AS c ON c.cliente_id = v.cliente_id
53 GROUP BY v.cliente_id, c.nombres
54 HAVING SUM(v.total_venta) = (
55     SELECT MAX(total)
56     FROM (
57         SELECT cliente_id, SUM(total_venta) As total
58         FROM Ventas
59         GROUP BY cliente_id
60     )
61 );
62
63
64
```

Vista de rejilla | Vista de formulario

Total rows loaded: 1

	cliente_id	nombres	total
1	1	Carlos Soto	1620000

Lección 6


SQLiteStudio (3.4.21)
Databases
name
Chinook_Sqlite (SQLite 3)
ABP 7 PROYECTO FIN...
Tables (3)
Clientes
Columns (8)
cliente_id
nombres
ciudad
region
genero
telefono
email
fecha_re...
Indexes
Triggers
Productos
Ventas
Views
SQL editor 1
ABP 7 PR(
Query
Historial
22
23 -- 4. Asignar stock inicial a cada producto
24 UPDATE Productos SET stock = 10 WHERE producto_id = 1;
25 UPDATE Productos SET stock = 20 WHERE producto_id = 2;
26 UPDATE Productos SET stock = 30 WHERE producto_id = 3;
27
28 -- 5. Registrar una nueva venta
29 INSERT INTO Ventas
30 (venta_id, cliente_id, producto_id, fecha_venta, cantidad, total_venta)
31 VALUES
32 (5, 3, 3, '2026-03-15', 1, 25000);
33
34 -- 6. Actualizar stock después de la venta
35 UPDATE Productos
36 SET stock = stock - 1
37 WHERE producto_id = 3;
38
39 -- 7. Verificar resultado final
40 SELECT * FROM Productos;
Vista de rejilla
Vista de formulario
Total rows loaded: 3
producto_ nombre_pı categoria precio_ur fecha_ingres stock
1 1 Notebook Tecnologia 800000 2026-01-15 10
2 2 Mouse Accesorios 15000 2026-01-15 20
3 3 Teclado Accesorios 30000 2026-01-15 30

B. Link del código subido a GitHub